

T-ESP-800 · EPITECH EIP

# Pawrise Care

Dossier de conception & documentation technique



## 0 Sommaire

|      |                                                   |    |
|------|---------------------------------------------------|----|
| 1    | Résumé exécutif .....                             | 4  |
| 2    | Vision & problématique .....                      | 5  |
| 2.1  | Le problème .....                                 | 5  |
| 2.2  | La solution Pawrise Care .....                    | 5  |
| 2.3  | Proposition de valeur .....                       | 5  |
| 2.4  | Personas .....                                    | 6  |
| 3    | Marché & analyse stratégique .....                | 7  |
| 3.1  | Proposition de valeur .....                       | 7  |
| 3.2  | Marché (TAM / SAM / SOM) .....                    | 7  |
| 3.3  | SWOT .....                                        | 8  |
| 3.4  | Matrice de confrontation .....                    | 8  |
| 3.5  | PESTEL .....                                      | 9  |
| 4    | Modèle économique & budget .....                  | 10 |
| 4.1  | Modèle économique .....                           | 10 |
| 4.2  | Unit economics (3 scénarios) .....                | 10 |
| 4.3  | Budget prévisionnel · collier (BOM) .....         | 11 |
| 4.4  | Coût annuel de l'infrastructure .....             | 11 |
| 5    | Exigences produit (PRD) .....                     | 14 |
| 5.1  | Exigences fonctionnelles (FR1 à FR43) .....       | 14 |
| 5.2  | Exigences non fonctionnelles (NFR1 à NFR10) ..... | 15 |
| 6    | WBS & fonctions par epic .....                    | 16 |
| 6.1  | User stories & critères d'acceptation .....       | 16 |
| 7    | Organisation & compétences .....                  | 18 |
| 7.1  | OBS · organisation des responsabilités .....      | 18 |
| 7.2  | Couverture nominative .....                       | 18 |
| 7.3  | Matrice de compétences (extrait) .....            | 18 |
| 7.4  | Justification de l'équipe .....                   | 19 |
| 8    | Matrice RACI .....                                | 20 |
| 8.1  | Règle appliquée .....                             | 20 |
| 8.2  | Version standard (A distribué) .....              | 20 |
| 8.3  | Version format suiveur (1 R / 1 A / 1 C) .....    | 20 |
| 9    | Planning & méthodologie .....                     | 21 |
| 9.1  | Vue d'ensemble .....                              | 21 |
| 9.2  | Diagramme de Gantt .....                          | 21 |
| 9.3  | Jalons .....                                      | 21 |
| 9.4  | Dépendances clés .....                            | 21 |
| 9.5  | Méthodologie .....                                | 21 |
| 10   | Plan qualité .....                                | 23 |
| 10.1 | Definition of Done .....                          | 23 |
| 10.2 | Stratégie de tests par couche .....               | 23 |
| 10.3 | Conventions de code .....                         | 23 |
| 10.4 | Workflow Git .....                                | 23 |
| 10.5 | CI/CD & déploiement .....                         | 24 |
| 10.6 | Onboarding développeur .....                      | 24 |
| 11   | Architecture technique .....                      | 25 |

|      |                                                   |    |
|------|---------------------------------------------------|----|
| 11.1 | Vue d'ensemble (simplifiée)                       | 25 |
| 11.2 | Choix techniques justifiés                        | 25 |
| 12   | Cloud & infrastructure                            | 26 |
| 12.1 | Principes directeurs                              | 26 |
| 12.2 | Dimensionnement (issu de l'architecture)          | 26 |
| 12.3 | Comparaison des options d'hébergement             | 26 |
| 12.4 | Souveraineté : pourquoi l'UE                      | 27 |
| 12.5 | Stratégie en deux temps                           | 27 |
| 12.6 | La stack d'infrastructure                         | 27 |
| 12.7 | Maîtrise des coûts en développement               | 28 |
| 13   | IoT · le collier connecté                         | 29 |
| 13.1 | Les défis à résoudre                              | 29 |
| 13.2 | Architecture en 3 zones                           | 29 |
| 13.3 | Composants embarqués                              | 29 |
| 13.4 | Capteurs : choix justifiés                        | 30 |
| 13.5 | Réseau & sécurité                                 | 30 |
| 13.6 | Logique embarquée : machine à états               | 30 |
| 14   | IA · le Care Engine                               | 31 |
| 14.1 | Pipeline LangGraph (6 nœuds)                      | 31 |
| 14.2 | Garde-fous (défense en profondeur, 3 couches)     | 31 |
| 14.3 | Périmètre                                         | 31 |
| 14.4 | Coût des modèles (cascade « un modèle par nœud ») | 31 |
| 14.5 | Décisions d'architecture (ADR)                    | 32 |
| 14.6 | Objectifs de succès (KPIs MVP)                    | 32 |
| 15   | Gestion des risques                               | 33 |
| 15.1 | AMDEC produit                                     | 33 |
| 15.2 | Risk Map projet                                   | 33 |
| 16   | Annexes                                           | 35 |
| 16.1 | Glossaire                                         | 35 |
| 16.2 | Références des décisions d'architecture (ADR)     | 35 |
| 16.3 | Sources & outils                                  | 35 |
| 16.4 | Prototype en ligne                                | 35 |

# 1 Résumé exécutif

**Pawrise Care** est un collier connecté qui suit en continu la santé et le bien-être d'un animal, l'explique au propriétaire via un assistant IA, et transmet au vétérinaire un dossier structuré et normalisé. Le positionnement est non négociable : **orienter, jamais diagnostiquer**.

Là où les acteurs établis (Tractive, Weenect, Invoxia) se concentrent sur la localisation, Pawrise Care ouvre la passerelle propriétaire vers vétérinaire que personne ne couvre : c'est une stratégie océan bleu sur un marché adressable de 26,5 M de chiens et chats en France.

**11**

epics

**66**

user stories (52 MVP)

**10**

membres · 5 pôles

**20 mois**

de roadmap

Ce dossier réunit l'ensemble des livrables de conception : vision et marché, modèle économique et budget, découpage (WBS) et organisation (OBS, RACI), planning et méthodologie, plan qualité, gestion des risques, et l'architecture technique complète (système, cloud, IoT, IA). Une maquette interactive du produit (architecture jouable, diagrammes et contenus) est par ailleurs consultable sur notre site [pawrise-care.com](https://pawrise-care.com).

## CE QUI DISTINGUE PAWRISE CARE

Une IA d'orientation conçue conforme dès l'origine (Code rural, AI Act, RGPD), un réseau vétérinaire réel, un modèle économique dont le coût vétérinaire est plafonné (une part fixe de l'abonnement finance le réseau : la dépense vétérinaire ne peut donc pas dépasser ce budget), et une architecture souveraine (UE) maîtrisée en coût.

## 2 Vision & problématique

### 2.1 Le problème

Les signaux de santé d'un animal de compagnie passent le plus souvent inaperçus jusqu'à ce que la situation devienne grave. Entre deux visites vétérinaires, le propriétaire n'a aucune visibilité continue sur l'état réel de son animal, et le vétérinaire reçoit un historique imprécis, reconstitué de mémoire. Les signaux faibles (baisse d'activité, troubles du sommeil, variations de constantes) sont donc détectés trop tard, souvent au stade critique.

Les colliers connectés ont beaucoup progressé : au-delà du GPS, plusieurs acteurs (Invoxia, Tractive) embarquent désormais des capteurs de santé (activité, fréquence cardiaque et respiratoire) et des alertes. Mais tous s'arrêtent à la donnée brute : ils mesurent et affichent des courbes, sans les expliquer ni accompagner le propriétaire. La valeur manquante n'est pas un capteur de plus, c'est l'interprétation.

### 2.2 La solution Pawrise Care

Pawrise Care ne cherche pas à gagner la course aux capteurs. Sa valeur unique est l'**explicabilité** : un assistant IA, le Care Engine, traduit les données du collier en langage clair pour le propriétaire, contextualise les alertes et répond à ses questions. Et quand une situation le justifie, un réseau de vétérinaires partenaires prend le relais avec un dossier déjà préparé. La chaîne de valeur est claire : le collier mesure, l'IA explique et oriente, le vétérinaire décide.

### 2.3 Proposition de valeur

Notre différenciation n'est pas le nombre de capteurs mais l'interprétation accompagnée, là où les concurrents s'arrêtent à la donnée brute. Elle repose sur deux piliers :

- **Explicabilité par l'IA** : l'assistant conversationnel explique l'état de l'animal en clair et contextualise les alertes. Une valeur difficile à copier car elle s'appuie sur un corpus vétérinaire validé et des garde-fous.
- **Relais vétérinaire** : dès qu'un avis médical devient nécessaire, un vétérinaire partenaire prend le relais avec un dossier normalisé prêt à l'emploi. Ce réseau est rémunéré par une part fixe des abonnements (pool), ce qui plafonne le coût vétérinaire par construction (voir modèle économique).

C'est un positionnement **océan bleu** : ne pas livrer la guerre des capteurs, mais ouvrir la passerelle propriétaire vers vétérinaire que les acteurs établis ne couvrent pas.

#### RÈGLE D'OR, NON NÉGOCIABLE

L'assistant IA n'établit **jamais** de diagnostic médical (Code rural, article L243-1) : il explique, oriente, et invite à consulter. C'est précisément le réseau vétérinaire qui prend le relais dès qu'un acte médical est requis. Toute situation ambiguë déclenche une orientation ou une escalade vers le vétérinaire, jamais une conclusion médicale.

## 2.4 Personas

Deux personas primaires, les deux faces de la plateforme, volontairement approfondis.

### Propriétaire d'animal

25 à 55 ans, attaché à son animal, sans formation vétérinaire, accès vétérinaire parfois difficile (zone rurale, planning saturé).

**Objectifs** : comprendre l'état de son animal au quotidien, agir au bon moment, éviter l'urgence.

**Frustrations** : données brutes illisibles, incertitude « est-ce grave ? », délais et coût des consultations.

### Vétérinaire partenaire

En cabinet ou en téléconsultation, prudent face aux outils IA généralistes, intéressé par la préparation en amont des consultations.

**Objectifs** : préparer ses consultations, gagner du temps, fidéliser sa clientèle.

**Frustrations** : historiques imprécis, données brutes non exploitables, sollicitations non filtrées.

**Segments prioritaires (early adopters)** : propriétaires en zone d'accès vétérinaire limité, foyers multi-animaux, propriétaires de races à risques de santé connus. Un acteur secondaire « système / administration » (provisioning des colliers, RGPD, exploitation) est traité dans les parties Organisation et Architecture.

## 3 Marché & analyse stratégique

### 3.1 Proposition de valeur

Les concurrents (Tractive, Weenect, Invoxia) ne se limitent plus au GPS : ils embarquent aussi des capteurs de santé. Mais ils s'arrêtent à la donnée brute. Pawrise Care **interprète** : GPS et capteurs de santé, plus une lecture accompagnée par l'IA et le vétérinaire pour orienter le propriétaire vers le bon soin, au bon moment. C'est un positionnement **océan bleu** : ne pas livrer la guerre des capteurs, mais ouvrir la passerelle propriétaire vers vétérinaire (dossier normalisé, suivi longitudinal) que les acteurs établis ne couvrent pas.

Trois piliers : **Prévention** (détecter les signaux faibles avant qu'ils n'empirent), **Orientation non-diagnostique** (aide à l'observation, jamais un acte médical), **Réseau vétérinaire** (escalade vers un professionnel avec un dossier prêt à l'emploi, rémunéré par une part fixe des abonnements).

### 3.2 Marché (TAM / SAM / SOM)

On dimensionne le marché en entonnoir : le **TAM** (Total Addressable Market) est le marché total si l'on captait tout le monde ; le **SAM** (Serviceable Addressable Market) la part que notre produit peut réellement adresser ; le **SOM** (Serviceable Obtainable Market) la part réalismement captable à court/moyen terme.

≈ 73 M

TAM · tous animaux de compagnie en France

≈ 26,5 M

SAM · chiens (9,9 M) + chats (16,6 M)

1 foyer / 2

possède au moins un animal

3-5 ans

SOM · dizaines de milliers d'abonnés

**TAM ≈ 73 M** : l'ensemble des animaux de compagnie en France, poissons, oiseaux et NAC compris. **SAM ≈ 26,5 M** : les chiens et chats (9,9 M + 16,6 M), seuls réellement adressables par un collier connecté. **SOM** : quelques dizaines de milliers d'abonnés visés sur 3 à 5 ans, compte tenu de la notoriété à bâtir, du budget et de la concurrence. Le repère « 1 foyer sur 2 » (part des foyers français possédant un animal) illustre la profondeur du marché. Le marché pet tech est en forte croissance, porté par l'humanisation des animaux ; les concurrents mesurent désormais la santé mais restent sur la donnée brute, l'interprétation accompagnée (IA + vétérinaire) reste notre différenciation.

Sources : [FACCO / Kantar, chiffres clés 2024-2025](#) ; rapports de marché pet tech ([Grand View Research](#), [pet wearables](#)).

### 3.3 SWOT

#### Forces

**Santé expliquée et accompagnée** : la donnée brute des concurrents traduite en clair par l'IA, relayée par un vétérinaire.

**Expertise IA + garde-fous non-diagnostiques** : RAG/LLM maîtrisés en interne, conformité intégrée dès le départ.

**Vétérinaire partenaire réel** : crédibilité scientifique et accès au métier.

**Données longitudinales propriétaires** : un actif santé par animal qui grossit avec le temps.

#### Faiblesses

**Hardware complexe et coûteux** : conception d'un collier fiable, lourde en temps et budget.

**Démarrage à froid des données** : fiabilité de l'IA dépendante de données à accumuler.

**Prototype non finalisé** : encore sur capteurs simulés, risque technique avant produit réel.

**Rentabilité à prouver** : modèle d'abonnement dont la viabilité reste à valider.

#### Opportunités

**Marché français massif et porteur** : 26,5 M de chiens et chats, segment connecté en croissance.

**Humanisation des animaux** : disposition réelle à payer pour la santé et la prévention.

**Assurance animale en croissance** : nos données aident la prévention et la tarification : canal + revenu B2B.

**Interopérabilité via le PDF normalisé** : gain de temps vétérinaire, adoption facilitée.

#### Menaces

**Concurrents déjà sur la santé** : Invoxia, Tractive mesurent déjà FC/FR : angle santé non vierge.

**Acteur capitalisé qui copie la couche vétérinaire** : un géant pourrait aller plus vite.

**Cadre réglementaire strict et mouvant** : acte médical réservé au vétérinaire, IA encadrée (AI Act).

**Adoption lente des vétérinaires** : profession prudente et surchargée.

Sources : marché FACCO / Kantar ; santé embarquée des concurrents [Invoxia](#), [Tractive](#) ; cadre réglementaire [AI Act \(UE 2024/1689\)](#), [Code rural](#), art. L243-1.

### 3.4 Matrice de confrontation

Ce qu'on fait du SWOT : croiser forces et faiblesses avec opportunités et menaces pour en tirer des axes d'action.

| Axe                                    | Stratégie                                                                                                                                                                                               |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Offensif · Forces × Opportunités       | Viser les assureurs animaliers : le suivi continu et le dossier vétérinaire réduisent leurs sinistres, nos données nourrissent leur tarification. On gagne un canal de distribution et un revenu B2B.   |
| Défensif · Forces × Menaces            | Ne pas livrer la guerre des capteurs (déjà gagnée par Tractive/Invoxia) : se protéger sur l'orientation accompagnée et le réseau vétérinaire, non copiables sans partenariats longs à nouer.            |
| Rattrapage · Faiblesses × Opportunités | Transformer le démarrage à froid en boucle de données : les premiers cabinets et assureurs partenaires apportent cas réels et co-financement, fiabilisant l'IA et le matériel plus vite.                |
| Vigilance · Faiblesses × Menaces       | Sécuriser les points faibles avant de grandir : LLM derrière une couche d'abstraction, garde-fous documentés pour encaisser un durcissement réglementaire, rentabilité prouvée sur un périmètre réduit. |



### 3.5 PESTEL

L'analyse macro-environnementale, fil conducteur : orienter, jamais diagnostiquer.

| Axe             | Synthèse                                                                                                                                           | À retenir                                              |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| Politique       | France 2030, BPI et UE financent l'IA et l'e-santé ; le bien-être animal est une priorité publique (loi du 30/11/2021) ; souveraineté des données. | Terrain favorable si hébergement souverain UE.         |
| Économique      | Marché animalier large et résilient ; pet tech en croissance. Vigilance : abonnement récurrent (~10 €/mois) dans un contexte de lassitude.         | Maîtrise du churn et du CAC décisive.                  |
| Socioculturel   | L'animal, membre de la famille ; demande de prévention proactive ; usages numériques entrés dans les mœurs.                                        | Demande qui colle à notre proposition.                 |
| Technologique   | Convergence capteurs / IA / cloud / télémédecine. Réserve : mesurer des constantes fiables sur un animal qui bouge.                                | Fiabilité de la mesure = risque central.               |
| Environnemental | Éco-conception imposée (Règlement Batteries UE 2023/1542, DEEE, indice de réparabilité) ; sobriété numérique (Green IT).                           | Durabilité matériel + sobriété numérique.              |
| Légal           | RGPD, Code rural (diagnostic réservé aux vétérinaires), AI Act, sécurité produit (GPSR UE 2023/988).                                               | Tout repose sur la frontière orienter / diagnostiquer. |

Sources officielles : France 2030, loi n° 2021-1539 (bien-être animal), FACCO / Kantar ; RGPD, Code rural L243-1, AI Act, GPSR (UE 2023/988), Règlement Batteries (UE 2023/1542), indice de réparabilité.

## 4 Modèle économique & budget

### 4.1 Modèle économique

Le hardware est vendu proche de son coût ; la rentabilité vient de l'**abonnement récurrent**.

Le modèle intègre un **pool de rémunération façon Spotify** : une **part fixe** de chaque abonnement alimente une cagnotte qui rémunère les vétérinaires, répartie au prorata de leur activité. Conséquence : le coût vétérinaire total ne peut jamais dépasser ce pourcentage du chiffre d'affaires, quel que soit le nombre de sollicitations. Il est donc plafonné « par construction » (mathématiquement), ce qui protège la marge : la dépense vétérinaire grandit avec les revenus, jamais plus vite. Des garde-fous (filtrage IA avant escalade, usage équitable) évitent l'abus.

**D'où vient le prix ?** On part pour l'instant sur **9,90 €/mois**, à faire évoluer au besoin. On se positionne volontairement dans le haut du marché (les concurrents facturent 5 à 13 €/mois) : notre valeur ajoutée (interprétation par l'IA et relais vétérinaire) justifie cet alignement par le haut.

**Les termes, en clair :**

- **Marge contributive** : ce qu'il reste de l'abonnement une fois payés les coûts directs d'un client (cloud, API d'IA, pool vétérinaire). Exemple : sur 9,90 €, s'il reste 6,80 €, la marge est de 69 %.
- **Churn mensuel** : le pourcentage de clients qui résilient chaque mois. S'il est de 3 %/mois, un client reste en moyenne  $1 / 0,03 \approx 33$  mois : c'est la **durée de vie client**.
- **LTV (valeur vie client)** : ce qu'un client rapporte au total sur toute sa durée de vie, soit abonnement  $\times$  nombre de mois  $\times$  marge. Exemple :  $9,90 \text{ €} \times 30 \text{ mois} \times 69 \% \approx 205 \text{ €}$ .
- **CAC (coût d'acquisition)** : ce qu'il faut dépenser (publicité, commercial) pour gagner un client.

### 4.2 Unit economics (3 scénarios)

Les **unit economics** répondent à une seule question : un client, à lui seul, rapporte-t-il plus qu'il ne coûte ? On encadre l'incertitude avec trois scénarios (optimiste, réaliste, pessimiste).

**Comment les scénarios sont bâtis.** On fixe trois **leviers d'entrée** à partir de repères du marché de l'abonnement grand public : le **churn** (taux de résiliation mensuel, 2 % / 3 % / 8 % par mois, d'où une durée de vie de 48 / 30 / 12 mois), le **CAC** (35 / 50 / 90 €, repères d'acquisition digitale) et la **marge** (78 / 69 / 35 %, selon ce que consomment cloud, IA et pool vétérinaire). Les deux dernières lignes, **LTV : CAC** et **Payback**, en sont ensuite calculées. Optimiste = meilleurs niveaux plausibles ; base = prudent et réaliste ; pessimiste = combinaison défavorable servant de garde-fou. C'est un modèle prévisionnel, à affiner avec les chiffres réels une fois en marché.

| Métrique                 | Optimiste              | Base                   | Pessimiste             |
|--------------------------|------------------------|------------------------|------------------------|
| Marge contributive (abo) | 78 %                   | 69 %                   | 35 %                   |
| Durée de vie (churn)     | $\approx 48$ mois      | $\approx 30$ mois      | $\approx 12$ mois      |
| CAC                      | $\approx 35 \text{ €}$ | $\approx 50 \text{ €}$ | $\approx 90 \text{ €}$ |
| LTV : CAC                | $\approx 10\times$     | $\approx 4\times$      | $\approx 0,5\times$    |
| Payback                  | $\approx 5$ mois       | $\approx 7$ mois       | non atteint *          |

**Comment lire. LTV : CAC** compare ce qu'un client rapporte (LTV) à ce qu'il coûte à acquérir (CAC) : au-dessus de  $3\times$  le modèle est sain, à  $1\times$  on est à l'équilibre, en dessous on perd de l'argent. **Payback** : le nombre de mois pour rembourser le coût d'acquisition grâce à la marge mensuelle. Lecture du scénario de base : un client rapporte environ  $4\times$  sa mise, remboursé en 7 mois. Le scénario pessimiste (churn et CAC élevés) ne rembourse jamais : c'est celui à éviter, en réduisant le churn et le coût d'acquisition.

### 4.3 Budget prévisionnel · collier (BOM)

Nomenclature du prototype, option retenue (LTE sur mesure). Les composants sont alignés sur la partie IoT.

| Composant                            | Coût unitaire |
|--------------------------------------|---------------|
| ESP32-S3 (MCU)                       | ≈ 9 €         |
| SIM7000E (modem LTE-M + GPS intégré) | ≈ 16 €        |
| IMU LSM6DSOX (accéléro + gyro)       | ≈ 4 €         |
| Capteur température TMP117           | ≈ 4 €         |
| Capteur cardiaque MAX30102 (option)  | ≈ 3 €         |
| Mémoire flash 128 Mo (SPI)           | ≈ 1,50 €      |
| Batterie LiPo 2000 mAh + PMU         | ≈ 12 €        |
| Boîtier étanche IP67                 | ≈ 2 €         |
| LED NeoPixel                         | ≈ 1 €         |
| PCB prototype                        | ≈ 0,50 €      |
| Carte SIM IoT 1NCE                   | ≈ 3 €         |

**Total unitaire : ≈ 56 € / unité**

### 4.4 Coût annuel de l'infrastructure

Coût d'exploitation sur **un an**, en production une fois le produit lancé, développement inclus. Hébergement souverain (UE), auto-géré via Terraform et Kubernetes. Le dimensionnement retenu (≈ 16 vCPU / 32 Go) découle directement de l'architecture (services Rust, bases de données, bus d'événements, broker MQTT, moteur IA, observabilité), justifié en partie Cloud.

**Hypothèses de calcul** : lancement à faible trafic et usage IA modéré ; prix serveurs = tarifs publics Hetzner de juin 2026 (ligne ARM CAX) ; parité euro/dollar retenue pour l'IA ; certificats TLS gratuits (Let's Encrypt).

#### Détail des postes

| Poste                                | Détail                                                                 | Coût annuel |
|--------------------------------------|------------------------------------------------------------------------|-------------|
| Cluster<br>Kubernetes<br>(auto-géré) | 1× CAX21 control-plane + 2× CAX31 workers (ARM Ampere, 8 vCPU / 16 Go) | ≈ 630 €/an  |
| Sauvegardes & volumes                | volumes bloc ~150 Go (0,057 €/Go) + backups automatiques (+20 %)       | ≈ 230 €/an  |
| Load balancer + IP publique          | LB11 (répartition de charge, TLS)                                      | ≈ 72 €/an   |
| Sauvegardes hors-site                | Storage Box (rétention longue durée)                                   | ≈ 60 €/an   |
| Modèle de langage (IA)               | Azure OpenAI + Cohere · ~0,03 à 0,05 €/conversation (variable)         | ≈ 600 €/an  |
| Nom de domaine                       | .com (TLS Let's Encrypt gratuit)                                       | ≈ 12 €/an   |
| Développement & tests                | dev en local (gratuit) + CI ; tests éphémères et IA à la consommation  | ≈ 150 €/an  |

**Total : ≈ 1 750 €/an**

Comment chaque poste est chiffré :

- **Compute (cluster)** :  $1 \times \text{CAX21} (10,49 \text{ €}) + 2 \times \text{CAX31} (2 \times 20,99 \text{ €}) = 52,47 \text{ €/mois}$ , soit  $\approx 630 \text{ €/an}$ . Taille calée sur le besoin de 16 vCPU / 32 Go.
- **Sauvegardes & volumes** : 150 Go de volumes bloc (0,057 €/Go) + sauvegardes automatiques (+20 % du prix des instances)  $\approx 19 \text{ €/mois}$ , soit  $\approx 230 \text{ €/an}$ .
- **Load balancer + IP publique** : LB11 à 5,99 €/mois, soit  $\approx 72 \text{ €/an}$ .
- **Sauvegardes hors-site** : Storage Box (rétention longue durée) 5 €/mois, soit  $\approx 60 \text{ €/an}$ .
- **Modèle de langage (IA)** : 0,03 à 0,05 € par conversation ; poste **variable**,  $\approx 600 \text{ €/an}$  à faible volume, qui croît avec les abonnements qui le financent.
- **Nom de domaine** :  $\approx 12 \text{ €/an}$  (TLS gratuit).

Sources des prix : [Hetzner Cloud](#) + ajustement du 15/06/2026 (CAX21 10,49 €, CAX31 20,99 €/mois ; volumes 0,057 €/Go) ; IA : [Azure OpenAI](#), [Cohere](#).

### Développement & tests (à la consommation)

L'essentiel du développement ne coûte rien, et les tests sont payés à l'usage :

- **Développement en local** : 0 €. Chaque développeur lance la stack sur sa machine (Docker Compose / kind).
- **Intégration continue** : GitHub Actions (offre incluse pour le projet).
- **Environnement de test / staging éphémère** : un petit cluster (31 €/mois en continu) n'est actif que 40 h/semaine, détruit le soir et le week-end via terraform destroy (24 % du temps), soit  $\approx 7,5 \text{ €/mois} \rightarrow \approx 90 \text{ €/an}$ .
- **Consommation IA de test** : 5 €/mois à l'usage, soit  $\approx 60 \text{ €/an}$ .

Total développement  $\approx 150 \text{ €/an}$ . Un serveur arrêté restant facturé chez Hetzner, l'économie vient de la destruction/recréation par IaC, pas de l'extinction (mécanisme détaillé en partie Cloud).

### Comparaison (coût annuel, capacité équivalente)

| Option d'hébergement             | Coût annuel*                                                  |
|----------------------------------|---------------------------------------------------------------|
| Bare metal (serveur dédié)       | $\sim 1\,000$ à $1\,500 \text{ €}$                            |
| Hetzner Cloud auto-géré (retenu) | $\approx 1\,750 \text{ €}$ (dont $\sim 150 \text{ €}$ de dev) |
| Cloud managé EU (Scaleway / OVH) | $\sim 3\,000$ à $3\,700 \text{ €}$                            |
| Hyperscaler (AWS / Azure / GCP)  | $\sim 6\,000$ à $10\,000 \text{ €}$                           |

\* Capacité équivalente (16 vCPU / 32 Go + bases de données, load balancer, sauvegardes et IA), faible trafic. Critères complets (souveraineté, exploitation, lock-in) en partie Cloud.

**Pourquoi pas le bare metal** : il reste acceptable pour des tests ponctuels, mais il n'est pas maintenable en production. Il ajoute toute une couche de complexité opérationnelle dont on ne veut absolument pas : gestion physique des serveurs, remplacement des disques et du matériel défaillant, redondance à provisionner et câbler en double, mises à jour firmware, supervision matérielle, et aucune élasticité pour absorber un pic ou créer un environnement à la demande. Le gain sur le compute brut ne compense pas cette charge d'exploitation permanente. On le réserve donc, au mieux, à des tests, jamais à la production.

**Pourquoi pas l'hyperscaler aujourd'hui** : notre charge de lancement ne le justifie tout simplement pas. Il coûte 4 à 6× plus cher, avec un fort lock-in et une souveraineté seulement partielle (le Cloud Act américain peut s'appliquer même sur une région européenne). Y aller avant d'en avoir le besoin serait du sur-engineering. On le garde en réserve pour un éventuel besoin multi-région ou un SLA entreprise.

**Retenu** : Hetzner Cloud auto-géré, le meilleur compromis coût / souveraineté / élasticité pour notre charge actuelle ; le cloud managé européen prend le relais au moment du passage à l'échelle.

### Prudence des chiffres (scénarios majorants)

Tous les montants présentés sont volontairement des majorants (hypothèses pessimistes), afin de ne jamais sous-estimer le budget :

- **Développement** : le calcul suppose un environnement de test actif environ 40 h par semaine, chaque semaine. En réalité, nous ne développerons pas en continu, et un environnement de staging ne sera monté que lors des vraies phases de test, bien plus rarement. Le coût réel de développement sera donc nettement inférieur aux  $\approx 150$  €/an affichés.
- **Intelligence artificielle** : le coût de 0,03 à 0,05 € par conversation est une fourchette haute, délibérément peu optimiste. Les optimisations prévues (mise en cache, cascade de modèles, modèles moins chers sur les nœuds simples) le feront baisser. Surtout, le poste IA est encadré par des **alertes de consommation** et des **plafonds journaliers** : au-delà d'un seuil, le service se dégrade gracieusement ou bascule sur une réponse de repli, ce qui rend tout dérapage budgétaire impossible.

Autrement dit, le total de  $\approx 1\,750$  €/an est un plafond prudent, pas une projection optimiste : le coût réel devrait être sensiblement inférieur.

### Ce qui varie avec l'échelle

Seul le poste IA est réellement variable : il croît avec le nombre de conversations, donc avec les abonnements qui le financent, ce qui garde ce coût couvert par la marge. Le compute, lui, augmente par paliers (ajout de workers) et bascule vers un cloud managé européen au moment du passage à l'échelle (voir partie Cloud). L'infrastructure reste ainsi maîtrisée et proportionnée à l'usage.

## 5 Exigences produit (PRD)

Ce chapitre constitue le cœur du **Product Requirements Document (PRD)** : il définit **ce que le produit doit faire et les qualités qu'il doit garantir**. La vision, le problème et les personas figurent en partie Vision ; les exigences ci-dessous ont été élicitées avec la méthode BMAD et servent de **référentiel** au découpage du travail (WBS, partie suivante). Chaque exigence est unique, testable, et tracée jusqu'aux epics qui la couvrent.

On distingue les **exigences fonctionnelles (FR)**, ce que le système doit faire, et les **exigences non fonctionnelles (NFR)**, les qualités et contraintes (fiabilité, sécurité, RGPD, performance, autonomie...).

### 5.1 Exigences fonctionnelles (FR1 à FR43)

Regroupées par domaine, elles décrivent les capacités attendues du système.

#### Collier & collecte

- FR1** Localisation GPS de l'animal
- FR2** Mesure de l'activité (accéléromètre)
- FR3** Mesure de la température corporelle
- FR4** Mesure du rythme cardiaque (option)
- FR5** Collecte périodique et temps réel
- FR6** Tampon local en cas de perte réseau
- FR7** Transmission sécurisée + gestion batterie

#### Backend & données

- FR8** Ingestion et normalisation des données
- FR9** Stockage en séries temporelles
- FR10** Calcul d'agrégats et d'indicateurs
- FR11** API de données pour l'application
- FR12** Comptes, animaux et règles d'accès

#### Moteur IA

- FR13** Prétraitement et fenêtrage des données
- FR14** Modèle de comportement normal par animal
- FR15** Détection d'anomalies (seuils adaptatifs)
- FR16** Assistant conversationnel (chat RAG)
- FR17** Escalade non-diagnostique vers le véto
- FR18** Rapport structuré (PDF / JSON)

#### App propriétaire

- FR19** Écran bien-être / état du jour
- FR20** Localisation temps réel et historique
- FR21** Zones de sécurité (geofencing) et alertes
- FR22** Notifications et rappels de soins
- FR23** Accès au chat IA et mise en relation véto

#### Vet Portal

- FR24** Vue patient (dossier regroupé)
- FR25** Historique médical chronologique
- FR26** Résumé contextualisé + données brutes
- FR27** Alertes, rapport et ajout de notes
- FR28** Export et suivi longitudinal

#### Cycle de vie & business

- FR29** Création de compte et onboarding
- FR30** Appairage du collier (BLE / QR code)
- FR31** Abonnement et facturation
- FR32** Support client et FAQ
- FR33** Profil animal, multi-animaux, remplacement
- FR34** Onboarding des vétérinaires partenaires

#### Téléconsultation & triage

- FR35** PDF normalisé pour le vétérinaire
- FR36** Handoff structuré IA vers vétérinaire
- FR37** Orientation non-diagnostique (triage)
- FR38** File de garde vétérinaire
- FR39** Prise de rendez-vous
- FR40** Pool de rémunération plafonné
- FR41** Garde-fous et contrôle qualité
- FR42** Routage vers le vétérinaire traitant
- FR43** Traçabilité et audit des téléconsultations

## 5.2 Exigences non fonctionnelles (NFR1 à NFR10)

Elles décrivent les qualités et contraintes transverses du système.

| Code                          | Exigence                                             |
|-------------------------------|------------------------------------------------------|
| NFR1 · Fiabilité              | Disponibilité du service et tolérance aux pannes.    |
| NFR2 · Autonomie / énergie    | Autonomie du collier et sobriété énergétique.        |
| NFR3 · Sécurité               | Chiffrement, authentification, mTLS de bout en bout. |
| NFR4 · Confidentialité / RGPD | Consentement, minimisation, droits des personnes.    |
| NFR5 · Éthique IA             | Non-diagnostic, garde-fous, transparence.            |
| NFR6 · Performance            | Temps de réponse maîtrisés (objectifs p95).          |
| NFR7 · Scalabilité / Ops      | Montée en charge et exploitabilité.                  |
| NFR8 · Robustesse physique    | IP67, résistance aux chocs et températures.          |
| NFR9 · Utilisabilité          | Simplicité d'usage et accessibilité.                 |
| NFR10 · Maintenabilité        | Qualité de code, tests, documentation.               |

## 6 WBS & fonctions par epic

Ce **Work Breakdown Structure (WBS)** décompose le **périmètre total** du projet de façon **hiérarchique et orientée livrables**, selon la méthode BMAD. Trois niveaux : le projet Pawrise Care (niveau 0), ses **11 epics** livrables (niveau 1, codes 1 à 11), et les **fonctions** attendues de chaque epic (niveau 2, codes 1.1, 1.2...). Les 66 user stories et leurs critères d'acceptation (Given/When/Then) affinent le niveau 2. Les codes FR/NFR affichés renvoient au référentiel défini en partie **Exigences produit (PRD)** ; la **traçabilité** epic vers exigences est portée par chaque carte ci-après (ligne « Exigences »).

Le diagramme unique ci-après (page suivante, en paysage) présente cette décomposition et, pour chaque livrable, décrit les **fonctions attendues** ainsi que les **exigences** qu'elles couvrent, afin d'en faciliter l'implémentation. Les codes FR/NFR renvoient au référentiel de la partie **Exigences produit (PRD)**. Périmètre MVP : 9 epics (52 US) ; post-MVP : epics 8 et 10.

### 6.1 User stories & critères d'acceptation

Chaque epic (niveau 1) se décline en **user stories** (66 au total) ; chacune porte des **critères d'acceptation** testables au format Given/When/Then, qui rendent la fonction vérifiable et prête à implémenter. Exemple :

#### STORY 1.1 · CRÉATION DE COMPTE SÉCURISÉE

**En tant que** propriétaire d'animal, **je veux** créer un compte avec email et mot de passe, **afin d'**accéder à l'application en toute sécurité.

**Given** un visiteur sur l'écran d'inscription

**When** il saisit un email valide et un mot de passe respectant la politique de sécurité

**Then** un compte est créé et un email de vérification est envoyé

**And** un mot de passe faible ou un email déjà utilisé déclenche une erreur explicite sans créer de compte.



WBS · décomposition hiérarchique orientée livrables

Niveau 0 : Pawrise Care (périmètre total) → Niveau 1 : 11 epics livrables (1 à 11) → Niveau 2 : fonctions (1.1, 1.2...). En italique : exigences (FR/NFR) couvertes.

|                                                                                                                                                                                                                                                                                                                                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div><div>1 · Onboarding &amp; appairage7 US · MVP</div><div><div>1.1</div>Compte sécurisé + authentification</div><div><div>1.2</div>Profil animal (race, âge, poids)</div><div><div>1.3</div>Appairage BLE du collier (QR code)</div><div><div>1.4</div>Gestion multi-animaux</div><div><div>Exigences : FR12, FR29, FR30, FR33</div></div></div> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 7 Organisation & compétences

### 7.1 OBS · organisation des responsabilités

Dix personnes réparties en cinq pôles de deux, sous le pilotage du Product Owner (Yassine El Gherrabi, qui cumule pilotage produit et contribution IA/Data).

| Pôle            | Membres              | Périmètre                                                     |
|-----------------|----------------------|---------------------------------------------------------------|
| Backend / API   | Hamid · Aaditya      | API, ingestion, base de données, authentification, Vet Portal |
| IoT / Hardware  | Cyril · Ibrahim      | Firmware, capteurs, simulateur, collier                       |
| IA / Data       | Nino · Yassine       | Care Engine, RAG, POCs, pipeline de données                   |
| Design / Mobile | Adam · Elarif        | UX/UI, app mobile, identité visuelle, étude de marché         |
| Cloud / DevOps  | Oumar · Abderrahmane | Cloud, CI/CD, monitoring, sécurité                            |

Elarif est rattaché au pôle Design/Mobile (binôme avec Adam sur l'application) mais intervient en transverse sur le Backend, la CI/CD, le marché et le lien vétérinaire.

### 7.2 Couverture nominative

Chaque domaine est porté par un ou plusieurs responsables nommés, avec un backup identifié pour assurer la continuité.

| Domaine            | Responsable(s)         | Backup                              |
|--------------------|------------------------|-------------------------------------|
| Product Ownership  | Yassine                | Elarif (transverse, vision produit) |
| IoT / Hardware     | Cyril, Ibrahim         | Hamid (support embarqué)            |
| Backend / API      | Hamid, Aaditya, Elarif | Yassine                             |
| IA / Data          | Nino, Yassine          | Hamid (pipelines Python)            |
| Mobile / Frontend  | Adam, Elarif           | Hamid, Aaditya                      |
| Design / UX        | Adam                   | Elarif (intégration front)          |
| Market / Business  | Adam, Elarif           | Yassine (PO)                        |
| Cloud / DevOps     | Oumar, Abderrahmane    | Elarif (CI/CD)                      |
| Réseau vétérinaire | Elarif, Nino           | Binôme (2 contacts vété distincts)  |

### 7.3 Matrice de compétences (extrait)

Niveaux : ■ expert · ■ intermédiaire · ■ débutant · ■ aucune expérience.

| Compétence               | Yassine | Elarif | Ibrahim | Cyril | Aaditya | Hamid | Nino | Adam | Oumar | Abderrahmane |
|--------------------------|---------|--------|---------|-------|---------|-------|------|------|-------|--------------|
| Rust                     | ■       | ■      | ■       | ■     | ■       | ■     | ■    | ■    | ■     | ■            |
| Python                   | ■       | ■      | ■       | ■     | ■       | ■     | ■    | ■    | ■     | ■            |
| Microcontrôleurs (ESP32) | ■       | ■      | ■       | ■     | ■       | ■     | ■    | ■    | ■     | ■            |
| Firmware C/C++ embarqué  | ■       | ■      | ■       | ■     | ■       | ■     | ■    | ■    | ■     | ■            |
| IA · NLP / LLM / RAG     | ■       | ■      | ■       | ■     | ■       | ■     | ■    | ■    | ■     | ■            |

| Compétence                | Yassine     | Elarif      | Ibrahim     | Cyril       | Aaditya     | Hamid       | Nino        | Adam        | Oumar       | Abderrahmane |
|---------------------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|--------------|
| React / Next.js           | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div>  |
| Docker / Conteneurisation | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div>  |
| CI/CD                     | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div>  |
| Architecture logicielle   | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div>  |
| UI / UX Design            | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div>  |
| Réseau vétérinaire        | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div> | <div></div>  |

### 7.4 Justification de l’équipe

L’équipe a été constituée pour que chaque membre couvre un besoin réel du projet (IoT, Backend, IA, Mobile, Design, Cloud, Business). Trois membres sont co-fondateurs et promoteurs du projet : Yassine, Ibrahim et Elarif. Les profils clés : Yassine (3 ans NLP/LLM en production → Care Engine), Cyril (le plus expérimenté en embarqué, STM32/ESP32, projets IoT persos), Ibrahim (fullstack évoluant vers l’IoT, LoRaWAN), Nino (data engineer, ETL), Adam (designer + commercial, 3 ans freelance), Oumar et Abderrahmane (binôme Cloud/DevOps, même socle Epitech).

## 8 Matrice RACI

La RACI attribue, pour chaque activité, un rôle à chaque pôle : **R** responsable (produit le livrable), **A** approuvateur (garant, valide), **C** consulté, **I** informé.

### 8.1 Règle appliquée

#### LA VRAIE RÈGLE RACI

Le seul impératif du standard est : **exactement un A (garant) par activité** et **au moins un R**. Les C et I sont en nombre libre. Un même pôle peut être A et R (noté A/R). Nous présentons deux lectures : une version standard (A distribué par pôle) et une version au format demandé en suivi (un R, un A, un C par activité).

### 8.2 Version standard (A distribué)

Chaque pôle est garant de son propre livrable ; le Product Owner reste garant du cadrage et de la gouvernance (RGPD, qualité).

| Activité                   | Pilotage (PO) | Fullstack | IoT | Design/Mobile | IA/Data | Cloud/Ops |
|----------------------------|---------------|-----------|-----|---------------|---------|-----------|
| Cadrage & spécifications   | A/R           | C         | C   | C             | C       | C         |
| API & ingestion données    | C             | A/R       | C   | I             | C       | C         |
| Collier & firmware         | C             | C         | A/R | I             | C       | I         |
| Application mobile         | C             | R         | I   | A/R           | C       | I         |
| Moteur IA / Care Engine    | C             | C         | I   | C             | A/R     | I         |
| Portail vétérinaire        | C             | A/R       | I   | C             | C       | I         |
| Infra / CI-CD / monitoring | I             | C         | I   | I             | I       | A/R       |
| RGPD & sécurité données    | A             | R         | C   | I             | C       | C         |
| Tests & qualité            | A             | R         | R   | R             | R       | R         |

### 8.3 Version format suiveur (1 R / 1 A / 1 C)

Exactement un R, un A et un C par activité, le reste en I. Plus stricte que le standard, mais toujours valide (un A unique, au moins un R). Le Product Owner est l'approuvateur unique.

| Activité                   | Pilotage (PO) | Fullstack | IoT | Design/Mobile | IA/Data | Cloud/Ops |
|----------------------------|---------------|-----------|-----|---------------|---------|-----------|
| Cadrage & spécifications   | A             | R         | I   | I             | C       | I         |
| API & ingestion données    | A             | R         | I   | I             | I       | C         |
| Collier & firmware         | A             | C         | R   | I             | I       | I         |
| Application mobile         | A             | C         | I   | R             | I       | I         |
| Moteur IA / Care Engine    | A             | C         | I   | I             | R       | I         |
| Portail vétérinaire        | A             | R         | I   | I             | C       | I         |
| Infra / CI-CD / monitoring | A             | C         | I   | I             | I       | R         |
| RGPD & sécurité données    | A             | R         | I   | I             | I       | C         |
| Tests & qualité            | A             | R         | I   | I             | C       | I         |

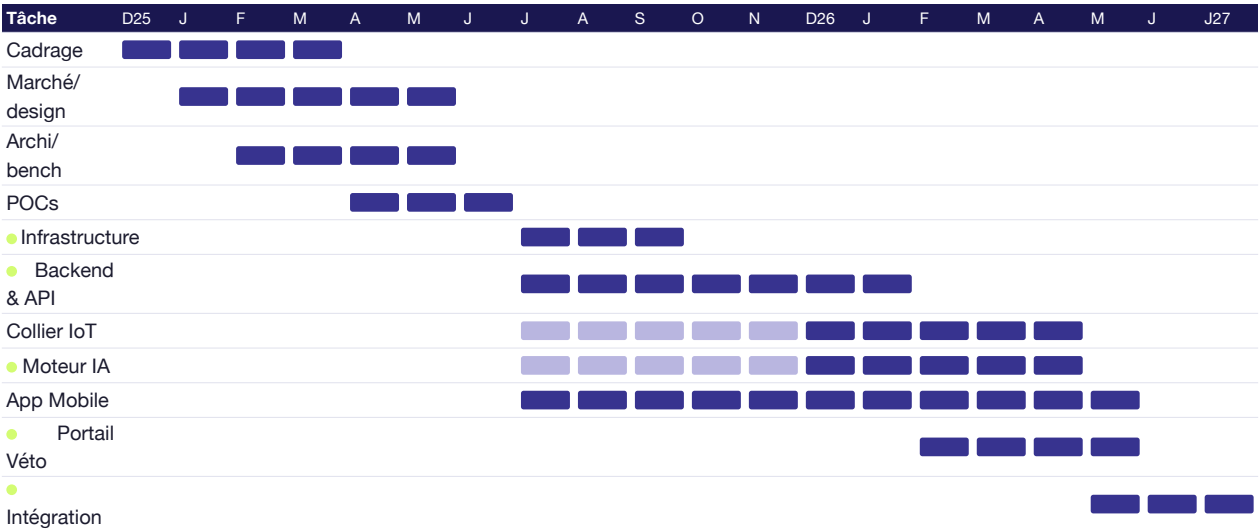
Les 6 colonnes recourent l'OBS : Pilotage (PO), Fullstack (backend & portail), IoT (collier), Design/Mobile (app), IA/Data (Care Engine), Cloud/Ops (infra).

## 9 Planning & méthodologie

### 9.1 Vue d'ensemble

Projet sur 20 mois (décembre 2025 → juillet 2027), en trois phases : **Conception** (livrables pour la keynote de juin 2026), **Développement** (tous les composants en parallèle), **Intégration** (assemblage, tests, stabilisation). Le parallélisme est possible car les composants sont volontairement peu couplés ; le portail vétérinaire est le seul vraiment dépendant (backend + IA), d'où un démarrage plus tardif.

### 9.2 Diagramme de Gantt



● chemin critique · zone claire = phase de dé-risquage (simulateur / POC) avant l'engagement du coûteux.

### 9.3 Jalons

| # | Date      | Jalon                                            |
|---|-----------|--------------------------------------------------|
| 1 | Juin 2026 | Keynote (tous les livrables de conception prêts) |
| 2 | Sep 2026  | Infrastructure opérationnelle                    |
| 3 | Nov 2026  | Simulateur collier                               |
| 4 | Jan 2027  | Backend complet                                  |
| 5 | Avr 2027  | MVP intégré (Care Engine + backend)              |
| 6 | Mai 2027  | Portail vétérinaire livré                        |
| 7 | Juil 2027 | Fin de projet (produit testé et stabilisé)       |

### 9.4 Dépendances clés

- **Infrastructure** → **Backend** : le backend se déploie sur l'infrastructure.
- **Simulateur collier** → **Moteur IA** : l'IA s'entraîne et se teste sur des données simulées.
- **Backend (API)** → **App Mobile** : l'app consomme les API au fil de l'eau.
- **Moteur IA + Backend** → **Portail Véto** : alertes et rapports viennent de l'IA et des données.

### 9.5 Méthodologie

**Scrum adapté** (agile itératif). Le périmètre évolue avec les apprentissages (POCs IA, faisabilité hardware, retours du vétérinaire partenaire) : on livre par incréments, on réoriente à chaque sprint. Sprints de 2 semaines, daily / point d'équipe, sprint review, rétrospective. Rôles : Product Owner

(Yassine), Scrum Master tournant, équipe par pôles. Outils : Jira (backlog, sprints, story points), Confluence (docs), GitHub (code, PR, CI), Figma (design).

**Alternatives écartées :** Waterfall (cahier des charges figé, incompatible avec l'incertitude R&D) ; Kanban pur (pas de cadence ni d'engagement de sprint, peu de prévisibilité pour 10 personnes) dont on retient le tableau visuel et la limite de WIP.

**Estimation :** Planning Poker sur échelle de Fibonacci (1, 2, 3, 5, 8, 13). Au-delà de 13 points, le ticket est découpé.

## 10 Plan qualité

Stratégie de tests par couche, conventions, workflow Git, CI/CD et onboarding : de quoi produire des livrables fiables et intégrer un nouveau développeur rapidement.

### 10.1 Definition of Done

Conditions qu'une User Story doit toutes remplir pour être considérée comme terminée :

- Les critères d'acceptation (Given/When/Then) sont tous satisfaits.
- Le code est relu et approuvé par au moins un pair via une Pull Request.
- Les tests de la story sont écrits et passent (unitaires et, si concernés, intégration).
- La CI est verte : lint et tests automatiques réussis (le merge est bloqué sinon).
- La documentation utile est à jour (README du service, doc d'API, Confluence si besoin).
- Aucune régression : un bug corrigé est couvert par un test de non-régression.

#### RÈGLE D'OR PRODUIT

L'IA n'émet jamais de diagnostic. Les tests du Care Engine vérifient que toute situation ambiguë déclenche une orientation ou une escalade vers le vétérinaire, jamais une conclusion médicale.

### 10.2 Stratégie de tests par couche

| Couche                           | Outils                                                      | Cible                                                                                    |
|----------------------------------|-------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Firmware collier (Rust no_std)   | Tests unitaires embarqués + simulateur de capteurs          | Logique de collecte/encodage validée hors matériel via le simulateur.                    |
| Services backend (Rust)          | Tests unitaires (cargo test) + tests d'intégration API      | Logique métier et endpoints couverts ; tests d'abus obligatoires sur l'authentification. |
| Moteur IA / Care Engine (Python) | pytest + jeux d'évaluation RAG + garde-fous anti-diagnostic | Non-régression des réponses et escalade systématique en cas de doute.                    |
| App mobile (Kotlin / SwiftUI)    | Tests unitaires natifs + tests d'UI critiques               | Onboarding, appairage et alertes vérifiés.                                               |
| Bout en bout (E2E)               | Scénarios end-to-end en phase d'intégration                 | Collier → backend → IA → app/portail validés ensemble.                                   |

### 10.3 Conventions de code

- Linting/formatage automatiques par langage : rustfmt + clippy (Rust), ruff/black (Python), ktlint (Kotlin), ESLint + Prettier (TS/Next).
- Nommage cohérent par langage (snake\_case Rust/Python, camelCase Kotlin/TS) et noms descriptifs.
- Toute fonctionnalité passe par une Pull Request relue par au moins un pair avant merge.
- Pas de secret en clair dans le code : variables d'environnement + secrets chiffrés (SOPS).

### 10.4 Workflow Git

| Branche      | Rôle                                                                      |
|--------------|---------------------------------------------------------------------------|
| main         | Branche stable et déployable. Protégée : merge uniquement via PR validée. |
| develop      | Intégration continue des fonctionnalités avant stabilisation.             |
| feat/<scope> | Une fonctionnalité = une branche, rattachée à un ticket Jira.             |

| Branche     | Rôle                                  |
|-------------|---------------------------------------|
| fix/<scope> | Correction de bug isolée et traçable. |

**Commits** conventionnels (feat:, fix:, docs:, chore:...) référençant la clé Jira. **Pull Request** : description claire, lien Jira, CI verte et revue d'au moins un pair avant merge.

## 10.5 CI/CD & déploiement

- À chaque Pull Request : lint + tests automatiques via GitHub Actions (merge bloqué si rouge).
- Build et publication d'images conteneurisées (Docker) versionnées.
- Déploiement GitOps : Helm + Argo CD synchronisent le cluster Kubernetes depuis Git.
- Secrets chiffrés (SOPS + age) ; observabilité OpenTelemetry → Prometheus/Loki/Tempo → Grafana.

## 10.6 Onboarding développeur

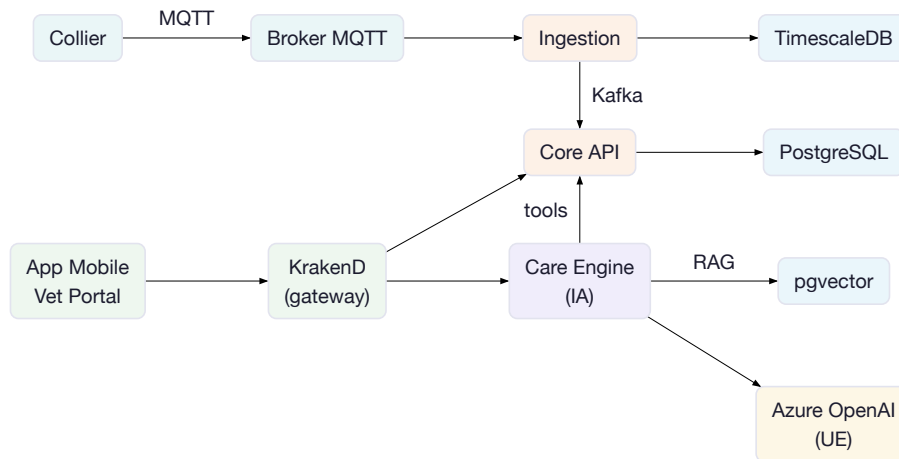
- Accès dépôt GitHub, projet Jira et espace Confluence dès l'arrivée.
- Lecture d'entrée : ce plan qualité, le Git Workflow et l'architecture globale.
- Mise en route locale : clone, installation, lancement en local (README de chaque service).
- Première contribution : un ticket « bonne première tâche », une PR suivant les conventions.



## 11 Architecture technique

### 11.1 Vue d'ensemble (simplifiée)

Le système s'organise en trois zones : clients et accès, services métier (Rust) autour d'un bus d'événements, et données. L'IA (Care Engine) est isolée et bornée. Schéma des flux principaux :



Version interactive complète (toutes les briques, flux et parcours) sur [pawrise-care.com](https://pawrise-care.com).

### 11.2 Choix techniques justifiés

Chaque décision structurante : besoin, alternatives écartées, décision, et trade-off assumé.

| Choix                                     | Alternatives écartées           | Décision & trade-off                                                                                                                  |
|-------------------------------------------|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Rust<br>(services métier)                 | Node, Python, Go, C#            | Perf + sûreté mémoire, empreinte minimale. Coût dev ~2,5× assumé (mitigé : monolithe-first, Python pour l'IA, réintroduction ciblée). |
| MQTT / EMQX<br>(entrée IoT)               | HTTP, CoAP, passerelle maison   | Standard IoT : pub/sub léger, QoS, offline gratuits. Un composant de plus à opérer.                                                   |
| Apache Kafka (bus)                        | RabbitMQ, appels directs        | Débit, rétention, découplage. Plus gourmand en RAM (dimensionne le cluster).                                                          |
| PostgreSQL<br>+ TimescaleDB<br>+ pgvector | Mongo + Influx + Qdrant séparés | Une seule famille : transactionnel, séries temporelles, vectoriel. Bascule Qdrant possible.                                           |
| Pipeline LangGraph<br>borné               | Agent LLM autonome              | Chaque étape testable, responsabilité juridique portée par le pipeline (ADR-001).                                                     |
| Azure OpenAI EU<br>+ cascade + Cohere     | LLM hors UE, modèle unique      | Data residency UE, ~0,03-0,05 €/conv. Dépendance Microsoft mitigée (interface LLMPProvider).                                          |
| Kubernetes auto-géré + GitOps             | PaaS, managé d'emblée           | Économique et souverain ; plus d'ops, mitigé par l'IaC. Managé EU au scale.                                                           |
| OIDC maison                               | Auth0, Keycloak, Cognito        | Souveraineté + défi formateur ; coûteux (~4-6 mois-homme), revue de sécurité obligatoire.                                             |

## 12 Cloud & infrastructure

Une infrastructure **souveraine** (UE / RGPD), **maîtrisée en coût** et **prête à scaler** : Hetzner auto-géré via Terraform aujourd'hui, cloud managé européen le jour où la charge le justifie. Décrite en code, déployée en GitOps. Cette partie justifie le dimensionnement (à partir de l'architecture), le choix d'hébergement (bare metal / Hetzner / managé / hyperscaler) et l'exigence de souveraineté.

### 12.1 Principes directeurs

- **Souveraineté & RGPD** : infrastructure et données de santé en UE ; Azure seulement pour le modèle de langage (région UE).
- **Infrastructure as Code** : toute l'infra en Terraform, un cluster se recrée à l'identique en une commande.
- **GitOps** : Git seule source de vérité ; Argo CD synchronise le cluster, rollback par revert.
- **Maîtrise des coûts** : lignes ARM, environnements de dev éphémères, auto-géré moins cher qu'un managé à capacités égales.
- **Observabilité** : chaque service tracé (OpenTelemetry) ; métriques, logs, traces centralisés dans Grafana.

### 12.2 Dimensionnement (issu de l'architecture)

Les besoins ne sont pas choisis au hasard : ils découlent des charges de travail décrites en partie Architecture (services Rust, bases de données, bus d'événements, broker MQTT, moteur IA, observabilité). Estimation à faible charge, pour un lancement :

| Charge de travail                                            | vCPU      | RAM     |
|--------------------------------------------------------------|-----------|---------|
| Services métier Rust (Core API, ingestion, auth, portail...) | ~2 à 3    | ~3 Go   |
| Bases de données (PostgreSQL + TimescaleDB + pgvector)       | ~2        | ~6 Go   |
| Bus d'événements (Kafka)                                     | ~1 à 2    | ~4 Go   |
| Broker MQTT (EMQX)                                           | ~1        | ~2 Go   |
| Care Engine (pipeline IA ; le LLM est déporté sur Azure)     | ~1 à 2    | ~3 Go   |
| Observabilité (Prometheus / Loki / Tempo / Grafana)          | ~1 à 2    | ~4 Go   |
| Gateway, ingress, Argo CD, PKI                               | ~1        | ~2 Go   |
| Total (avec marge)                                           | ≈ 16 vCPU | ≈ 32 Go |

Rust et l'absence de LLM local (déporté sur Azure) gardent l'empreinte basse. La cible ≈ 16 vCPU / 32 Go correspond à **2× CAX31** (2 × 8 vCPU / 16 Go) en workers, plus **1× CAX21** en control-plane. On monte en charge simplement en ajoutant des workers.

### 12.3 Comparaison des options d'hébergement

À capacité équivalente (≈ 16 vCPU / 32 Go + bases de données, load balancer, sauvegardes et IA, faible trafic) :

| Option                           | Coût annuel*      | Souveraineté UE          | Exploitation                                    | Lock-in                    |
|----------------------------------|-------------------|--------------------------|-------------------------------------------------|----------------------------|
| Bare metal (serveur dédié)       | ~1 000 à 1 500 €  | Oui                      | Élevée (matériel, pannes, redondance à doubler) | Aucun                      |
| Hetzner Cloud auto-géré (retenu) | ≈ 1 600 €         | Oui                      | Moyenne (K8s, mais tout en IaC)                 | Aucun (Terraform portable) |
| Cloud managé EU (Scaleway / OVH) | ~3 000 à 3 700 €  | Oui                      | Faible (control-plane managé)                   | Modéré (fournisseur)       |
| Hyperscaler (AWS / Azure / GCP)  | ~6 000 à 10 000 € | Partielle (Cloud Act US) | Faible                                          | Fort                       |

\* Ordres de grandeur, faible trafic et usage IA modéré. Prix serveurs : tarifs publics Hetzner juin 2026 (Cloud CAX et dédiés AX).

Le **bare metal** est le moins cher en compute brut mais impose l'exploitation matérielle (remplacement de disque, redondance physique à acheter en double) et n'offre aucune élasticité : incompatible avec des environnements de dev éphémères. L'**hyperscaler** est confortable mais 4 à 6× plus cher, avec un fort lock-in et une souveraineté seulement partielle (voir ci-dessous). Le **cloud managé européen** est la bonne cible au scale. Pour la phase actuelle, **Hetzner Cloud auto-géré** offre le meilleur compromis coût / souveraineté / flexibilité.

## 12.4 Souveraineté : pourquoi l'UE

Les données de santé animale, rattachées à un propriétaire identifié, sont des données personnelles au sens du RGPD. Un hébergeur européen évite l'exposition au **Cloud Act** américain, qui peut contraindre un fournisseur US à communiquer des données même hébergées dans une région UE. D'où le choix d'un acteur européen (Hetzner) pour toute l'infrastructure ; le seul service non européen, le grand modèle de langage, passe par **Azure OpenAI en région UE**, avec résidence des données et sans réutilisation pour l'entraînement (voir partie IA).

### DÉCISION

Cluster Kubernetes sur **VPS Cloud Hetzner (lignes ARM CAX)**, auto-géré et provisionné par Terraform. Depuis la hausse Hetzner du 15 juin 2026 (jusqu'à +176 % sur l'AMD, +30 % seulement sur l'ARM), la ligne ARM est la plus rentable et suffit à notre charge. Bare metal écarté (exploitation matérielle) ; cloud managé européen gardé pour le passage à l'échelle.

## 12.5 Stratégie en deux temps

**Maintenant (dev → keynote → premiers utilisateurs)** : cluster Kubernetes provisionné par Terraform (control-plane CAX21 + 2 workers CAX31), déploiement GitOps, secrets chiffrés, PKI interne, souveraineté UE.

**Au scale (traction réelle)** : Cloud managé EU, Scaleway Kapsule (control-plane managé gratuit, PostgreSQL managé, Object Storage, GPU à la demande), sans lock-in hyperscaler. Alternative : OVHcloud Managed Kubernetes. Azure AKS gardé pour un besoin multi-région ou un SLA entreprise.

## 12.6 La stack d'infrastructure

| Techno    | Rôle            | Pourquoi                                                                                                      |
|-----------|-----------------|---------------------------------------------------------------------------------------------------------------|
| Terraform | Provisionnement | Décrit et crée toute l'infra Hetzner de façon reproductible ; détruire/recréer un environnement à la demande. |

| Techno                               | Rôle               | Pourquoi                                                                                         |
|--------------------------------------|--------------------|--------------------------------------------------------------------------------------------------|
| Kubernetes                           | Orchestration      | Fait tourner et auto-répare la vingtaine de services ; redémarrage, placement, montée en charge. |
| Helm                                 | Packaging          | Chaque service en chart versionné et paramétrable (dev/prod).                                    |
| Argo CD                              | Déploiement GitOps | Déploie depuis Git ; l'état du cluster suit le dépôt, rollback par revert.                       |
| SOPS + age                           | Secrets            | Secrets chiffrés dans Git, déchiffrés uniquement dans le cluster.                                |
| step-ca                              | PKI interne        | Autorité de certification : chaque collier et service s'authentifie en mTLS.                     |
| OTel + Prometheus/Loki/Tempo/Grafana | Observabilité      | Métriques, logs et traces unifiés.                                                               |
| KrakenD + Traefik / cert-manager     | Gateway & Ingress  | Agrégation d'API, entrée HTTP et TLS automatique.                                                |

## 12.7 Maîtrise des coûts en développement

Un serveur arrêté reste facturé chez Hetzner : « éteindre » n'économise rien. Le bon mécanisme s'appuie sur l'IaC :

- Facturation horaire avec plafond mensuel : un serveur qui n'existe que quelques heures par jour est facturé au prorata.
- Environnements de dev éphémères : `terraform destroy` le soir et le week-end, `terraform apply` le matin.
- Un cluster de dev actif 40 h/semaine (au lieu de 168) coûte 25 % du plafond, soit près de 70 % d'économie sur le compute de dev.
- Données préservées sur volumes persistants ( 0,057 €/Go/mois) ou snapshots ; seul le compute est recréé.

## 13 IoT · le collier connecté

Le collier est le cœur produit : capteurs embarqués, connectivité cellulaire basse consommation et firmware Rust, pensés pour tenir sur batterie, fonctionner en zone blanche et transmettre en sécurité.

**7 à 15 j**

autonomie cible

**IP67**

poussière + immersion

**-20 à +60°C**

plage de température

**2000 mAh**

batterie Li-Po

### 13.1 Les défis à résoudre

- **Autonomie énergétique** · cible 7 à 15 jours ; GPS, modem et capteurs sont énergivores, d'où veille profonde, capteurs à la demande, transmission par batch.
- **Zones blanches** · le cas critique (fugue) arrive souvent sans couverture ; mode dégradé : stockage local puis rattrapage (store and forward).
- **Précision de localisation** · GPS qui dérive et premier fix long, accélérés par l'A-GPS et une fréquence adaptée au contexte.
- **Robustesse** · chocs, eau, température : certification IP67 et composants qualifiés -20 à +60°C, gestion thermique au contact de la peau.
- **Sécurité des données** · un appareil qui transmet la position doit être inviolable : authentification par certificat et chiffrement de bout en bout.

### 13.2 Architecture en 3 zones

- **Zone 1 · Collier (Edge)** : MCU, capteurs, modem, GPS, batterie. Collecte, tampon local, transmission chiffrée.
- **Zone 2 · Réseau & communication** : LTE-M + SIM IoT, transport MQTT sur TLS, pensé basse consommation et réseau instable.
- **Zone 3 · Backend & cloud** : ingestion, stockage, analyse (stack du site : Hetzner/Kubernetes, observabilité Grafana). Le collier s'y authentifie en mTLS.

### 13.3 Composants embarqués

| Composant                    | Choix                       | Rôle                                                                                                             |
|------------------------------|-----------------------------|------------------------------------------------------------------------------------------------------------------|
| Microcontrôleur              | ESP32-S3 ou nRF9160         | Exécute le firmware Rust, pilote la machine à états, orchestre capteurs et modem ; clé privée en secure element. |
| Modem cellulaire             | LTE-M (SIM7000E / nRF9160)  | Connectivité basse consommation ; transmission de la télémétrie.                                                 |
| GPS / GNSS                   | NEO-M8N ou GPS intégré      | Localisation ; A-GPS pour accélérer le premier fix.                                                              |
| Centrale inertielle          | LSM6DSOX (ST)               | Accéléromètre + gyroscope : activité, immobilité, mouvements brusques.                                           |
| Température                  | TMP117 (TI)                 | Température corporelle (précision $\pm 0,1^\circ\text{C}$ ).                                                     |
| Fréquence cardiaque (option) | MAX30102 (PPG)              | Rythme cardiaque par photopléthysmographie.                                                                      |
| Mémoire flash                | 128 Mo (SPI)                | Stockage local en attente de transmission.                                                                       |
| Batterie & PMU               | Li-Po 2000 mAh + fuel gauge | Alimentation autonome, gestion de charge, niveau précis.                                                         |

### 13.4 Capteurs : choix justifiés

- **Activité (IMU)** → **LSM6DSOX** : très basse consommation, fonctions intelligentes embarquées qui déchargent le MCU.
- **Température** → **TMP117** : précision médicale ( $\pm 0,1^{\circ}\text{C}$ ), idéale pour détecter une fièvre.
- **Fréquence cardiaque** → **MAX30102 (PPG)** : meilleur compromis précision/consommation ( 20 mW), bibliothèque mature. Le radar 60 GHz est écarté (consommation 4×, précision faible sur chien mobile, usage réel = maison fixe).

### 13.5 Réseau & sécurité

**Réseau** : LTE-M plutôt que NB-IoT (garde la mobilité et une latence faible, indispensables pour suivre un animal) ; transport MQTT sur TLS (broker EMQX : pub/sub léger, QoS, offline gratuits) ; SIM IoT 1NCE ; mode dégradé store-and-forward.

**Sécurité** : PKI X.509 (Root CA → CA intermédiaire signant colliers et serveur, via step-ca) ; authentification mutuelle mTLS (un appareil non enregistré est refusé, même firmware reverse-engineered) ; provisioning en usine (device\_id + certificat, clé en secure element) ; activation par appairage BLE (QR code).

### 13.6 Logique embarquée : machine à états

Le firmware Rust adapte acquisitions et transmission selon l'état de l'animal et du réseau : **BOOT** (démarrage), **PAIRING** (appairage BLE), **REST** (repos, sobre), **ACTIVE** (activité), **ESCAPE** (sortie de zone, GPS accéléré), **LIVE** (temps réel à la demande), **NO\_NETWORK** (zone blanche, stockage local), **LOW\_POWER** (batterie faible, services réduits).

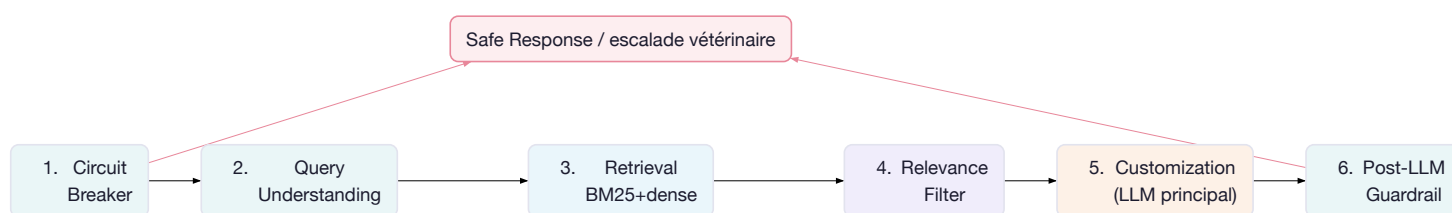
## 14 IA · le Care Engine

L'assistant conversationnel contextualise l'état d'un chien à partir des données du collier, répond sur la base d'un corpus vétérinaire validé, et organise un handoff structuré vers un vétérinaire quand c'est pertinent.

### RÈGLE D'OR

L'assistant n'établit jamais de diagnostic médical (Code rural, article L243-1) : il explique, oriente, et invite à consulter. Architecture : un pipeline LangGraph déterministe borné à 6 nœuds, déployé sur Azure OpenAI Europe ; la mémoire long-terme est déléguée au backend via tool calls.

### 14.1 Pipeline LangGraph (6 nœuds)



Pipeline déterministe et borné : chaque étape est testable et journalisée (audit append-only, rétention 5 ans). Le LLM est cantonné à sa tâche.

### 14.2 Garde-fous (défense en profondeur, 3 couches)

- **Couche 1 · pré-LLM (Circuit Breaker)** : classifier d'intention ; tout ce qui n'est pas « clean » est dérouter vers une Safe Response ou une escalade, sans atteindre le LLM principal.
- **Couche 2 · raisonnement** : system prompt strict + whitelist de tools + obligation de citer le corpus pour tout claim médical.
- **Couche 3 · post-LLM** : détection « diagnostic-like », contrôle d'ancrage sur les sources, déclenchement forcé de l'escalade selon des règles explicites.

### 14.3 Périmètre

**Ce qu'on fait** : Q&A santé/comportement (corpus + télémétrie), contextualisation d'une alerte collier, handoff vétérinaire structuré (PDF + JSON), refus systématique du diagnostic, audit complet.

**Ce qu'on ne fait jamais** : diagnostic explicite, prescription/posologie, gestion d'urgences vitales (redirection urgentiste), conseils de dressage, conseils nutritionnels précis.

### 14.4 Coût des modèles (cascade « un modèle par nœud »)

Chaque nœud utilise le modèle le moins cher qui fait le travail ; seul le nœud de génération mobilise le gros modèle.

| Nœud                    | Modèle                                   | Prix (entrée / sortie, 1M tokens) |
|-------------------------|------------------------------------------|-----------------------------------|
| 1 · Circuit Breaker     | mini · GPT-5-nano                        | 0,05 \$ / 0,40 \$                 |
| 2 · Query Understanding | mini · GPT-5-nano                        | 0,05 \$ / 0,40 \$                 |
| 3 · Retrieval           | embeddings text-embedding-3-large + BM25 | 0,13 \$ / 0                       |
| 4 · Relevance Filter    | Cohere Rerank 3.5 multilingue            | 2 \$ / 1 000 recherches           |
| 5 · Customization       | principal · GPT-5 (ou GPT-4.1)           | ~1,25 à 2,50 \$ / ~10 \$          |
| 6 · Post-Guardrail      | mini · GPT-5-nano                        | 0,05 \$ / 0,40 \$                 |

**COÛT PAR CONVERSATION ≈ 0,03 À 0,05 €**

Répartition : génération 85 %, reranking 10 %, nœuds mini 2 %, embeddings < 0,5 %. La cascade économise 30 à 35 % vs tout passer par le gros modèle. Ce coût valide la cible de la spec (moins de 0,05 €/conversation). Alternative souveraine prévue (ADR-002) : Mistral (UE), coût équivalent, données 100 % UE. Prix éditeurs en dollars, parité dollar/euro retenue.

**14.5 Décisions d'architecture (ADR)**

| ADR                             | Décision                                                                                           |
|---------------------------------|----------------------------------------------------------------------------------------------------|
| 001 · Pipeline borné            | 6 nœuds déterministes, pas d'agent autonome : responsabilité juridique portée par le pipeline.     |
| 002 · Azure OpenAI EU + cascade | Data residency UE, pas d'entraînement sur les données ; interface LLMProvider (basculer possible). |
| 003 · Reranker dédié            | Cohere Rerank 3.5 : latence ~10× meilleure qu'un filtre LLM, meilleure pertinence.                 |
| 004 · Mémoire = backend         | Pas de fact store dans le chatbot ; tool calls vers le Core API (source de vérité unique).         |
| 005 · Sortie JSON structurée    | Le front affiche les actions sans parser du markdown fragile.                                      |
| 006 · RGPD progressif           | MVP : masquage regex des PII ; V1 : Presidio + dictionnaire vétérinaire.                           |
| 007 · Garde-fous 3 couches      | Défense en profondeur : l'échec d'une couche n'expose pas le système.                              |
| 008 · Retrieval hybride         | BM25 (termes exacts) + dense (similarité), fusion RRF.                                             |
| 009 · Query Understanding       | Reformulation en termes vétérinaires + tool télémétrie avant le RAG.                               |
| 010 · Audit append-only         | Log immuable de chaque conversation, rétention 5 ans : défense juridique et base d'eval.           |

**14.6 Objectifs de succès (KPIs MVP)**

0 faux diagnostic sur le golden set · ≥ 95 % de citation quand claim médical · 5 à 15 % de taux d'escalade approprié · réponse complète p95 < 8 s · recall@5 ≥ 0,85 · détection jailbreak ≥ 95 %.



## 15 Gestion des risques

La maîtrise des risques repose sur deux analyses complémentaires : une **AMDEC** centrée sur le produit (modes de défaillance techniques) et une **Risk Map** centrée sur le projet (probabilité contre impact).

### 15.1 AMDEC produit

L'AMDEC évalue chaque mode de défaillance par son indice de priorité de risque **IPR = G × O × D** (Gravité × Occurrence × Détection, chaque facteur de 1 à 10). Les modes sont triés par IPR décroissant : la priorisation se lit de haut en bas.

| Code | Sous-système              | Mode de défaillance                             | G  | O | D | IPR | Niveau   |
|------|---------------------------|-------------------------------------------------|----|---|---|-----|----------|
| H03  | Capteur température / FC  | Mesures erronées (faux positifs physiologiques) | 7  | 7 | 6 | 294 | Critique |
| I01  | Analyse comportementale   | Faux négatif : anomalie non détectée            | 9  | 5 | 6 | 270 | Critique |
| I03  | Chat RAG vétérinaire      | Réponse médicalement incorrecte du LLM          | 9  | 5 | 5 | 225 | Critique |
| E01  | Déclenchement automatique | Escalade non déclenchée pour un cas urgent      | 10 | 4 | 5 | 200 | Critique |
| I02  | Analyse comportementale   | Faux positif : alerte infondée                  | 5  | 7 | 5 | 175 | Critique |
| H01  | Batterie                  | Décharge rapide / autonomie insuffisante        | 7  | 6 | 4 | 168 | Critique |
| E02  | Portail vétérinaire       | Vétérinaire partenaire indisponible             | 8  | 5 | 4 | 160 | Critique |
| T03  | GPS WebSocket             | Latence excessive du GPS en temps réel          | 6  | 5 | 5 | 150 | Critique |
| H04  | Communication BLE         | Perte de synchronisation smartphone ↔ collier   | 5  | 7 | 4 | 140 | Critique |
| H02  | Capteurs                  | Défaillance du capteur accéléromètre            | 6  | 4 | 5 | 120 | Critique |
| E03  | Continuité des données    | Données collier non accessibles au vétérinaire  | 7  | 4 | 4 | 112 | Critique |
| T01  | API Backend               | Indisponibilité du backend (downtime)           | 8  | 4 | 3 | 96  | Élevé    |
| T02  | Time-series DB            | Corruption ou perte de données capteurs         | 8  | 3 | 4 | 96  | Élevé    |
| I04  | Chat RAG vétérinaire      | Indisponibilité ou latence excessive du LLM     | 6  | 4 | 3 | 72  | Élevé    |

Seuils : IPR < 30 faible · 30–59 modéré · 60–99 élevé · ≥ 100 critique. La détection est inversée (1 = très détectable, 10 = indétectable).

### 15.2 Risk Map projet

La Risk Map positionne chaque risque projet selon son score **Score = P × I** (Probabilité × Impact, de 1 à 5). Registre trié par score décroissant.

| ID  | Catégorie | Risque                                                          | P | I | Score | Niveau   |
|-----|-----------|-----------------------------------------------------------------|---|---|-------|----------|
| R01 | Technique | Capteur FC non fiable sur l'animal                              | 4 | 5 | 20    | Critique |
| R02 | Technique | Bus factor : Cyril seul sur le hardware IoT (firmware)          | 4 | 5 | 20    | Critique |
| R07 | Légal     | LLM génère un conseil médical erroné : responsabilité juridique | 3 | 5 | 15    | Critique |
| R03 | Technique | Données d'entraînement insuffisantes : modèle IA peu fiable     | 3 | 4 | 12    | Élevé    |
| R04 | Technique | GPS : latence / dérive, temps réel non garanti                  | 3 | 4 | 12    | Élevé    |

| ID  | Catégorie | Risque                                                      | P | I | Score | Niveau |
|-----|-----------|-------------------------------------------------------------|---|---|-------|--------|
| R09 | Marché    | Vétérinaire partenaire unique : rupture du partenariat      | 3 | 4 | 12    | Élevé  |
| R12 | Humain    | Départ ou indisponibilité d'un membre clé                   | 3 | 4 | 12    | Élevé  |
| R13 | Planning  | Sous-estimation de la charge firmware : dérapage calendaire | 4 | 3 | 12    | Élevé  |
| R05 | Technique | BLE : perte de connexion collier-mobile                     | 3 | 3 | 9     | Élevé  |
| R14 | Planning  | Retard du prototype collier (composants / impression 3D)    | 3 | 3 | 9     | Élevé  |
| R06 | Technique | Dépassement du budget API LLM                               | 4 | 2 | 8     | Modéré |
| R08 | Légal     | Non-conformité RGPD (données de santé animale)              | 2 | 4 | 8     | Modéré |
| R10 | Marché    | Faible adoption : propriétaires pas prêts à payer           | 2 | 4 | 8     | Modéré |
| R11 | Marché    | Concurrence déjà positionnée (Tractive, Weenect)            | 4 | 2 | 8     | Modéré |

Zones : Score ≥ 15 critique · 9–14 élevé · 5–8 modéré · 1–4 faible.

#### POLITIQUE DE TRAITEMENT

Un plan de mitigation est rédigé pour chaque risque dont le score est supérieur ou égal à 9, rattaché aux actions de l'AMDEC et aux jalons de dé-risquage du planning. Les risques sous ce seuil sont surveillés sans plan dédié.

## 16 Annexes

### 16.1 Glossaire

| Terme             | Définition                                                                                               |
|-------------------|----------------------------------------------------------------------------------------------------------|
| Care Engine       | Moteur d'IA conversationnelle de Pawrise Care (pipeline LangGraph 6 nœuds).                              |
| RAG               | Retrieval-Augmented Generation : le LLM répond à partir de documents récupérés (corpus vétérinaire).     |
| mTLS              | Authentification mutuelle TLS : client et serveur se prouvent mutuellement leur identité par certificat. |
| GitOps            | Déploiement piloté par Git : l'état du cluster suit ce qui est décrit dans le dépôt (Argo CD).           |
| IaC               | Infrastructure as Code : l'infrastructure décrite en fichiers versionnés (Terraform).                    |
| IPR               | Indice de priorité de risque (AMDEC) = Gravité × Occurrence × Détection.                                 |
| LTV / CAC         | Valeur vie client / coût d'acquisition client.                                                           |
| LTE-M             | Réseau cellulaire basse consommation pour l'IoT, avec mobilité.                                          |
| Store and forward | Stockage local des données hors réseau, renvoyées à la reconnexion.                                      |

### 16.2 Références des décisions d'architecture (ADR)

Les dix ADR du Care Engine sont détaillées dans la partie « IA · le Care Engine ». Les choix d'architecture système et cloud sont justifiés dans les parties correspondantes (besoin, alternatives écartées, décision, trade-off).

### 16.3 Sources & outils

- **Marché** : [FACCO / Kantar 2024-2025](#), [Grand View Research](#) (pet wearables).
- **Prix cloud** : [Hetzner Cloud](#) (ajustement tarifaire du 15 juin 2026), [Scaleway Kapsule](#), [OVHcloud Managed Kubernetes](#).
- **Prix modèles IA** : [Azure OpenAI](#), [Cohere](#), [Mistral](#) (tarifs 2026).
- **Cadre légal** : [Code rural](#), art. L243-1, [RGPD](#), [AI Act](#), [GPSR \(UE 2023/988\)](#), [Règlement Batteries \(UE 2023/1542\)](#).
- **Outils projet** : [Jira](#), [Confluence](#), [GitHub](#), [Figma](#). **Documentation** : [Typst](#) (tableaux générés depuis la base de contenu du prototype).

### 16.4 Prototype en ligne

Une version interactive de l'architecture (cockpit jouable), des diagrammes et de l'ensemble des contenus est disponible sur [pawrise-care.com](https://pawrise-care.com). Ce dossier en est la version académique, imprimable et autoportante.